

Robots in Context

Lecture 1

Leon Bodenhagen lebo@mmmi.sdu.dk

Associate Professor

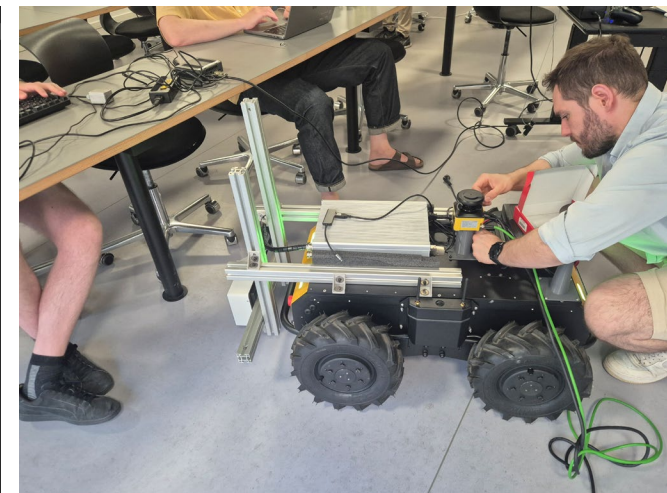
SDU Robotics, Maersk Mc-Kinney Moller Institute

Robots in Context – Lecture 1

Agenda for today:

- General introduction to the course
- Introduction to ROS

Background



History of the Course

- Robots in Context aka Introduction to Robotics
 - 5 ECTS module within a mandatory 15 ECTS course (AI + VIS + ROB)
 - 5 ECTS standalone elective course (ROB)
- (joint) project (pass/fail) as access requirement for the oral exam
- Combined workload was to the high side

Robots in Context today

- This course is combined:
 - 7,5 ECTS as a mandatory ROB course (alongside 7,5 ECTS for AI, no VIS)
 - Still 5 ECTS standalone elective course
- The content has not changed
 - The 5 ECTS version has one topic less (parts of localization)
 - In both cases practical elements from the course will be part of the exam
- I cannot see who is enrolled in which part of the course
 - At the exams it is evident though

Teaching staff

- Main responsible:
 - Leon Bodenhagen
 - LEBO@mmmi.sdu.dk
- Instructor
 - Rasmus Gjerlund Kaae Christiansen
 - RASCH@mmmi.sdu.dk
- It is best to catch us during breaks or after lectures, otherwise communication via email (preferred instead of itsLearning)

Topics covered

(minor adjustments might occur)

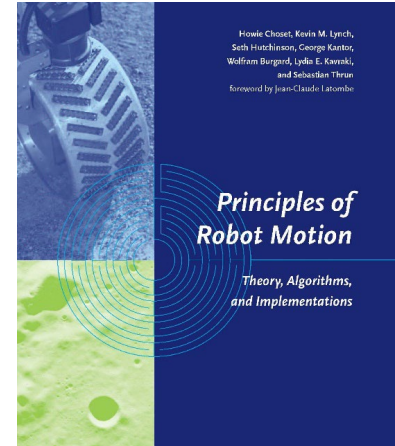
- ROS (today)
- Model-free planning algorithm (Bug, next week)
- Sensors and Feature extraction
- Point to point planning
- Coverage planning / cell decomposition
- Sampling based planning
- Localization
 - KF, EKF, PF
 - (+ ROS Navigation Stack)
- Gazebo + ROS continued
- Small project / bigger programming exercise

Course structure

- 1 Lecture / week
 - 'Weekly note' available on itslearning
 - Student presentations
 - Discussing students' solutions from last week's exercises
 - Introduction of new content
 - Exercise support
- Programming exercises
 - Meant to be worked on at a lower frequency
 - Typically spans a 2+ weeks
 - Selected students present their solutions in class
 - All students present at least once throughout the course
 - Plenty of exercises will be available, i.e., 1 per week
 - The exam involves a presentation too – you can use the programming exercise as a free, low-stake rehearsal.

Material for the course

- Main textbook:
(available at the bookstore)
- For ROS:
The best source is currently the ROS website:
<https://docs.ros.org/en/jazzy/Concepts.html>
- Additional material on ItsLearning



Principles of Robot Motion: Theory, Algorithms, and Implementations

H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard

MIT Press 2005

Introduction to ROS

Why ROS and why in this course?

The take-home message today:

Overview & terminology

Being able to apply basic concepts

You need to explore it yourself actively





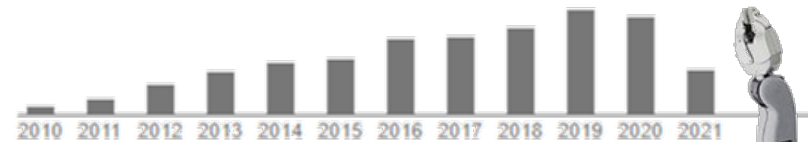
History of the Robot Operating System (ROS)

- Open source project, started in 2007 with Stanford
- Mentioned in the literature in 2009:

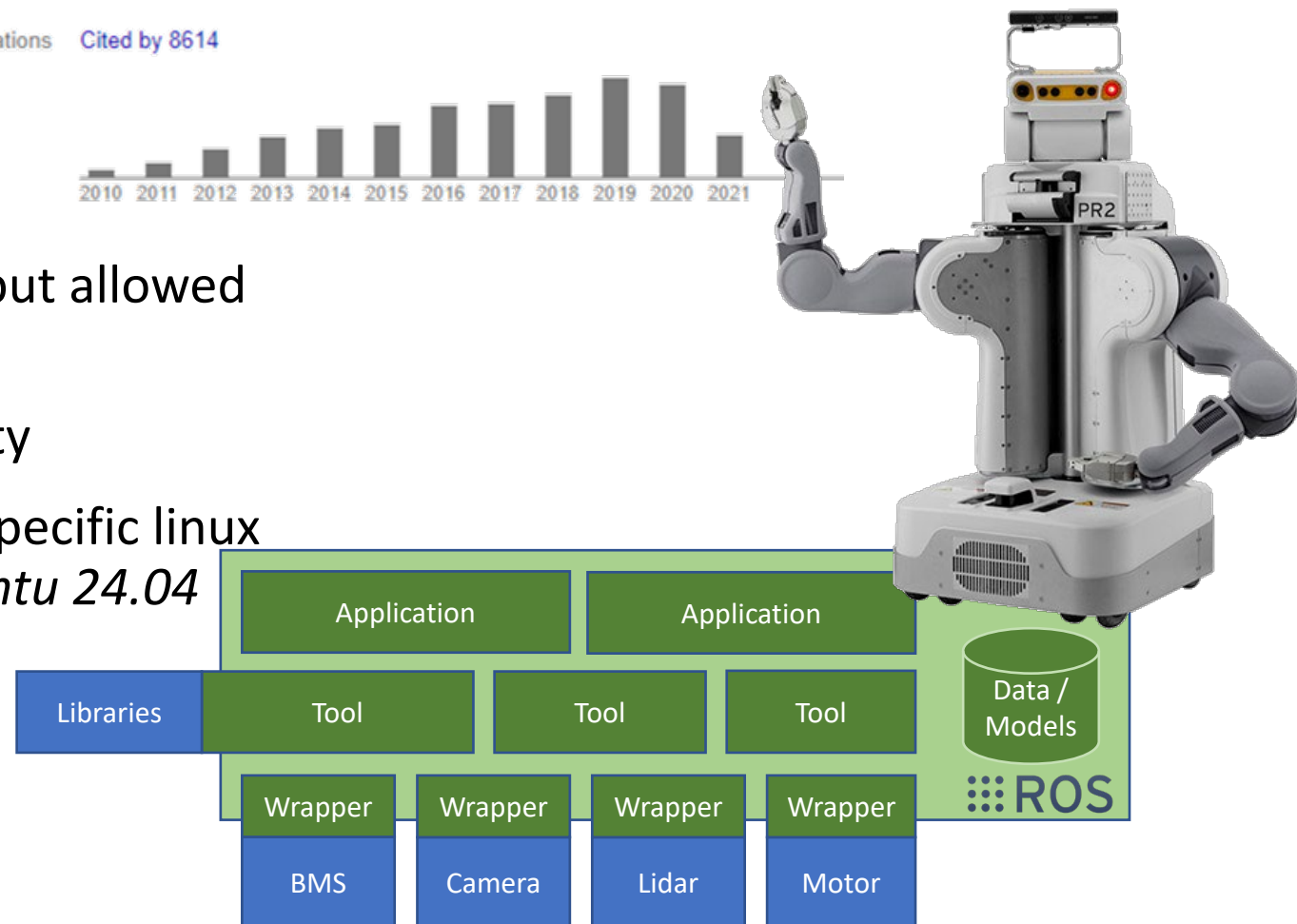
ROS: an open-source Robot Operating System

M Quigley et al. ICRA workshop on open source software, 2009

Total citations Cited by 8614



- Before: Proprietary solutions have been used but allowed a.o. for limited code reuse only
- Now: quasi standard in the research community
- Different versions of ROS are typically tied to specific linux versions, i.e. *ROS2 Jazzy Jalisco* goes with *Ubuntu 24.04*
 - ROS1: No support for MS Windows,
ROS2: various platforms incl. MS Windows



History of the Robot Operating System (ROS)

- 'Plumbing'

Processes management, drivers, inter-process communication

- Tools

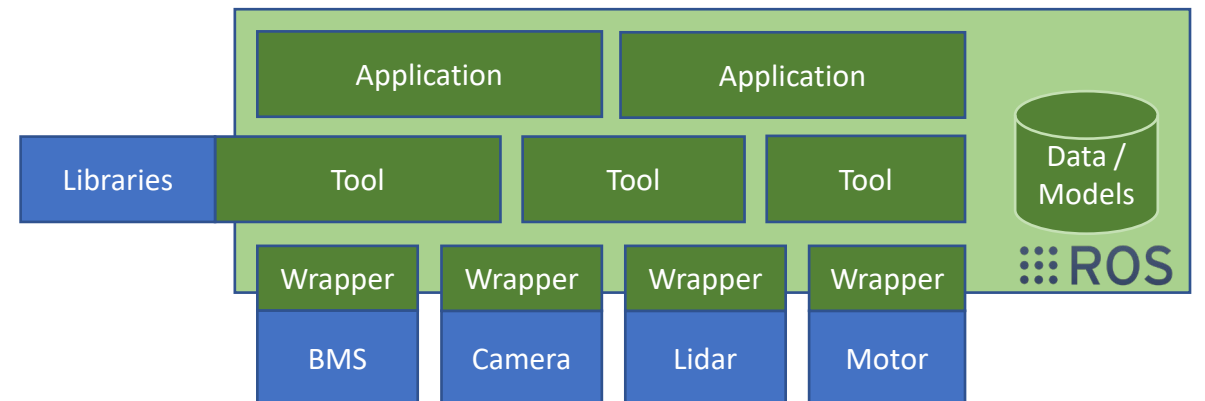
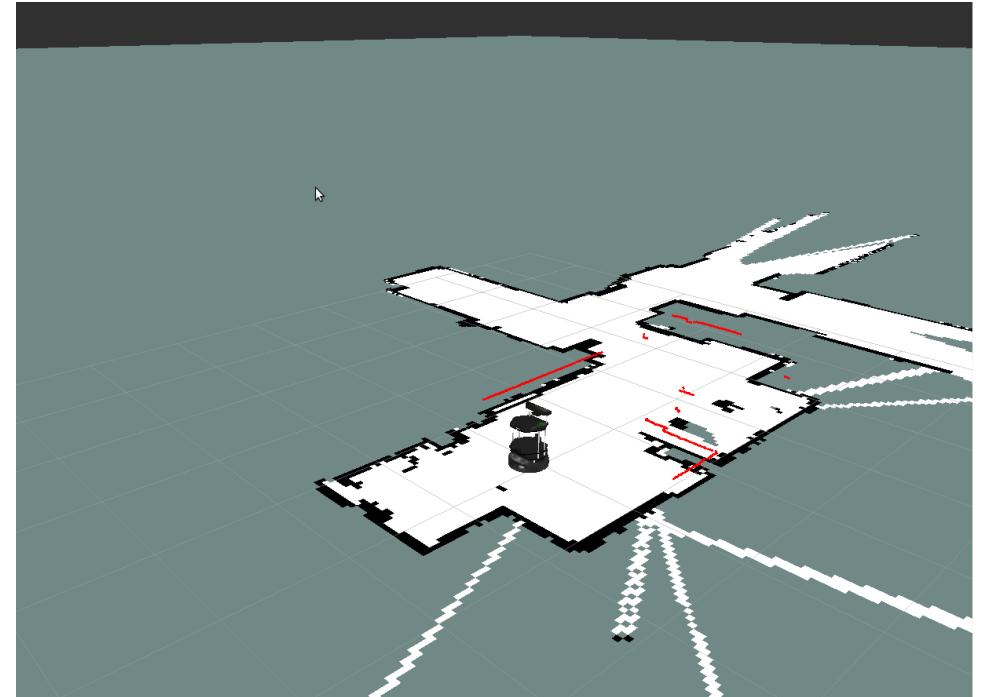
Simulation, Visualization, Data logging

- Capabilities

Control, Planning, Mapping

- Ecosystem

Package organization, Software distribution

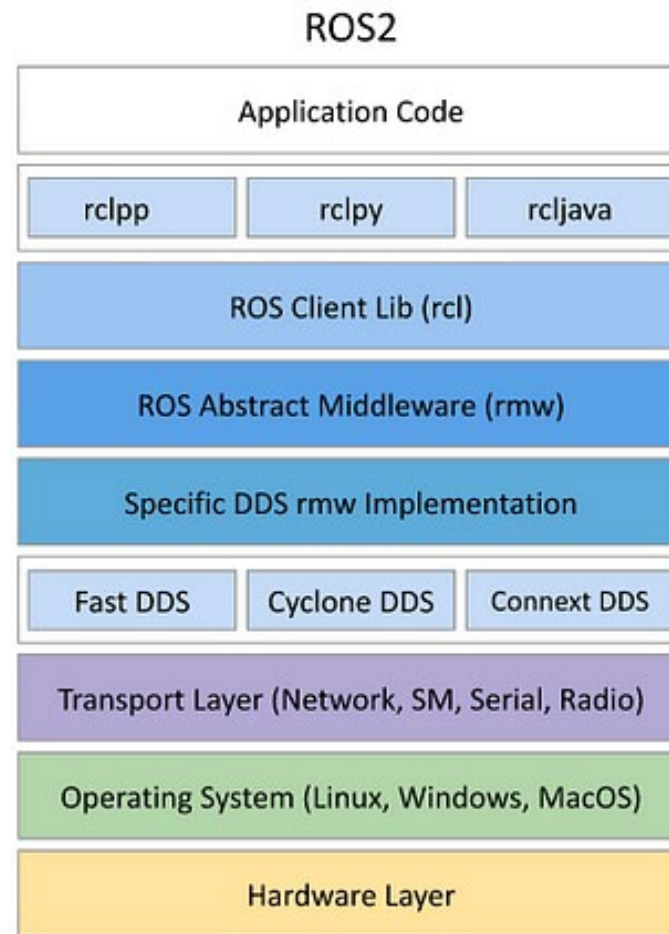
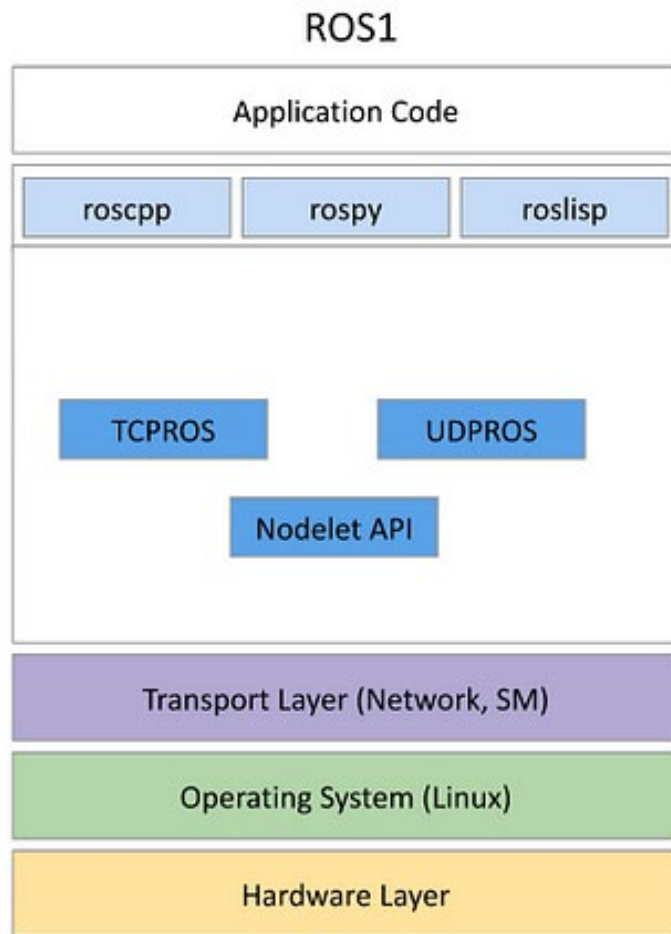




General Philosophy in ROS

- **Peer to peer**
Potentially many individual programs communicate via defined API (ROS messages, services, etc.).
- **Distributed**
Programs can run on multiple computers and communicate over the network.
- **Multi-lingual**
ROS modules can be written in various languages (C++, Python, MATLAB, Java, etc.).
- **Light-weight**
Stand-alone libraries are wrapped with a thin ROS layer.
- **Shared**
Most ROS software is accessible, open-source and free to use.

ROS1 vs. ROS2



There are many differences, one quite essential is:

- **ROS1** required a unique Master
- **ROS2** has no master and uses DDS (Data Distribution Service) instead.
'Domain ID' moderates connectivity.

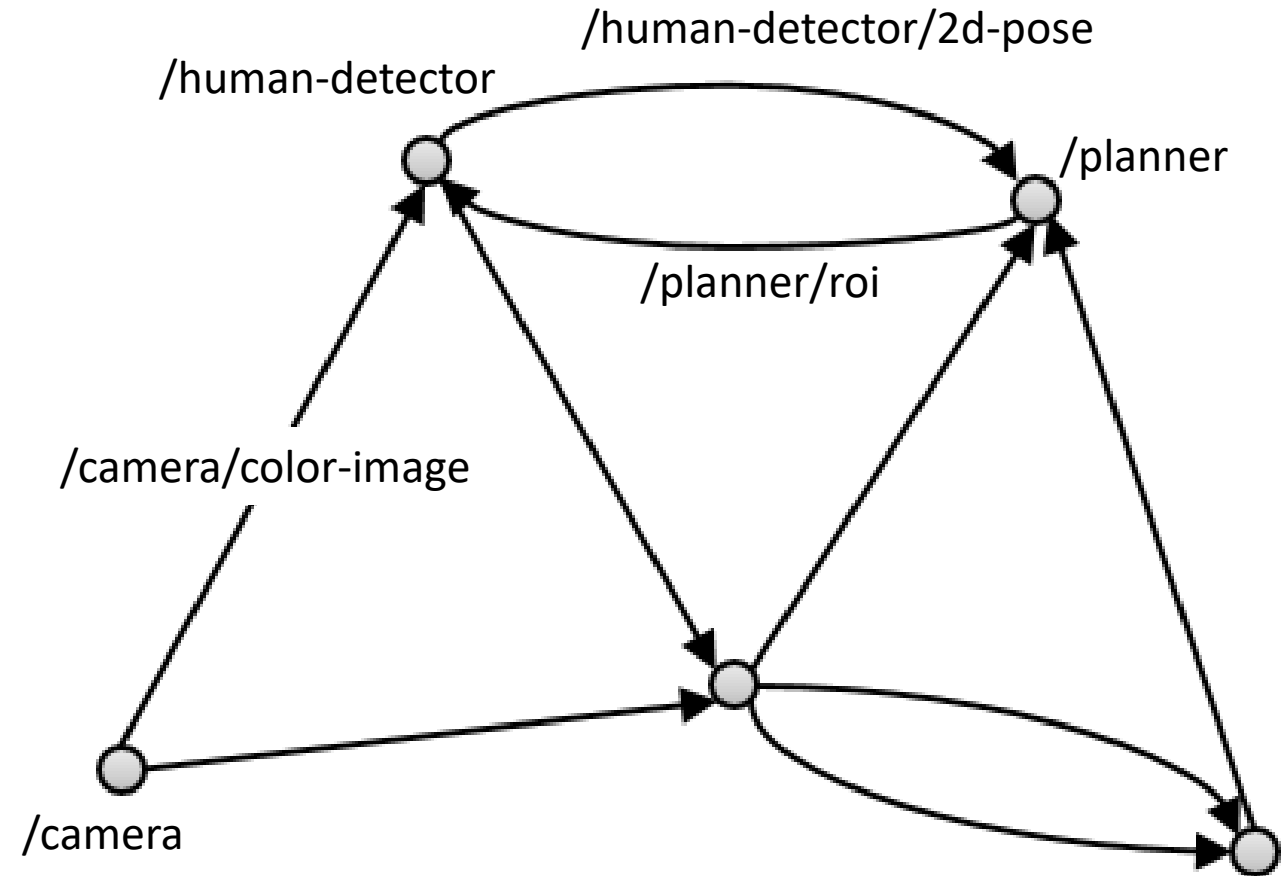
Robot Operating System (ROS)

Systems can in general be understood as graphs

Nodes are running processes

Edges are information being exchanged

Can be explored at runtime:
`rqt-graph`



Workspace, Build tools, launching nodes

- Build tools: *colcon*
 - Underneath: CMake or Python
 - Compile: `colcon build`

- ROS Packages

```
ros2 pkg create --build-type ament_cmake --node-name my_node my_package
```

- Naming convention: lower_case_w_underscores
- A package includes at least:

- Package.xml

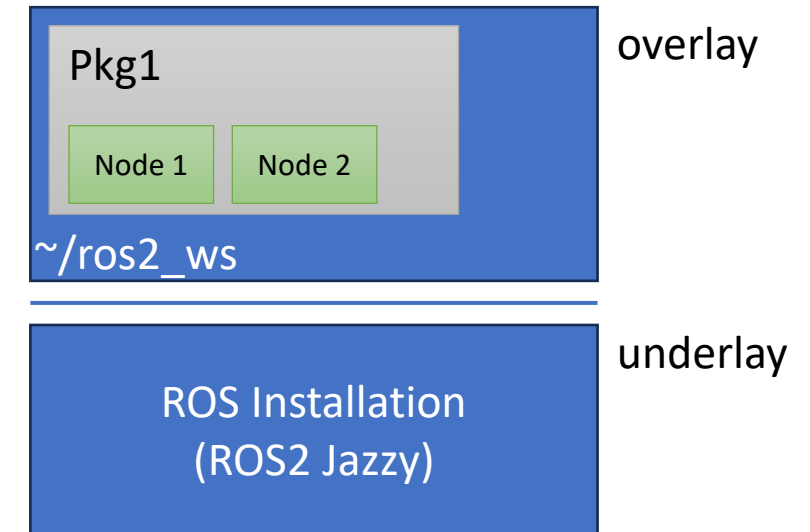
Meta information: version, author, dependencies etc.

- CMakeLists.txt

The Information required for the compiler

- /src

Your code goes here



Workspace, Build tools, launching nodes

- Build tools: *colcon*
 - Underneath: CMake or Python
 - Compile: `colcon build`

- ROS Packages

`ros2 pkg create --build-type ament_cmake --node-name my_node my_package`

- Naming convention: lower_case_w_underscores
- A package includes at least:

- Package.xml

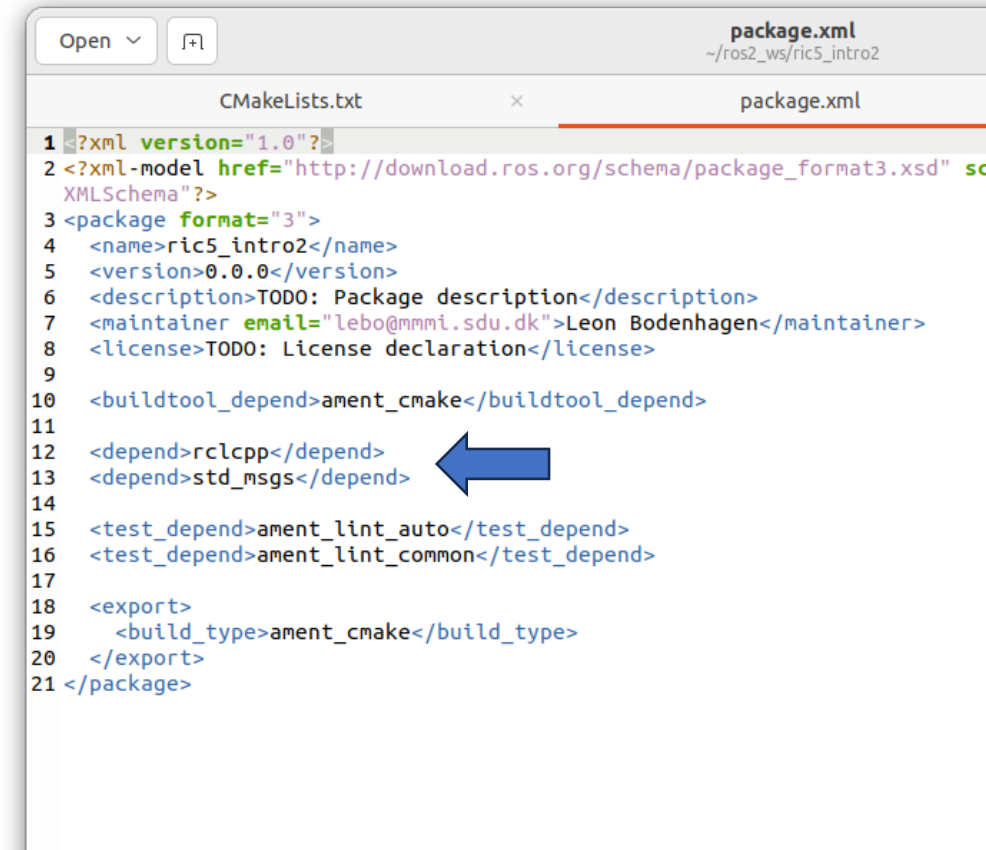
Meta information: version, author, dependencies etc.

- CMakeLists.txt

The Information required for the compiler

- /src

Your code goes here



```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schemaLocation="http://download.ros.org/schema/package_format3.xsd" type="xsl"?>
3 <package format="3">
4   <name>ric5_intro2</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="lebo@mmmi.sdu.dk">Leon Bodenhagen</maintainer>
8   <license>TODO: License declaration</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <depend>rclcpp</depend>
13  <depend>std_msgs</depend>
14
15  <test_depend>ament_lint_auto</test_depend>
16  <test_depend>ament_lint_common</test_depend>
17
18  <export>
19    <build_type>ament_cmake</build_type>
20  </export>
21 </package>
```

Workspace, Build tools, launching nodes

- Build tools: *colcon*
 - Underneath: CMake or Python
 - Compile: **colcon build**

- ROS Packages

`ros2 pkg create --build-type ament_cmake --node-name my_node my_package`

- Naming convention: lower_case_w_underscores
- A package includes at least:

- Package.xml

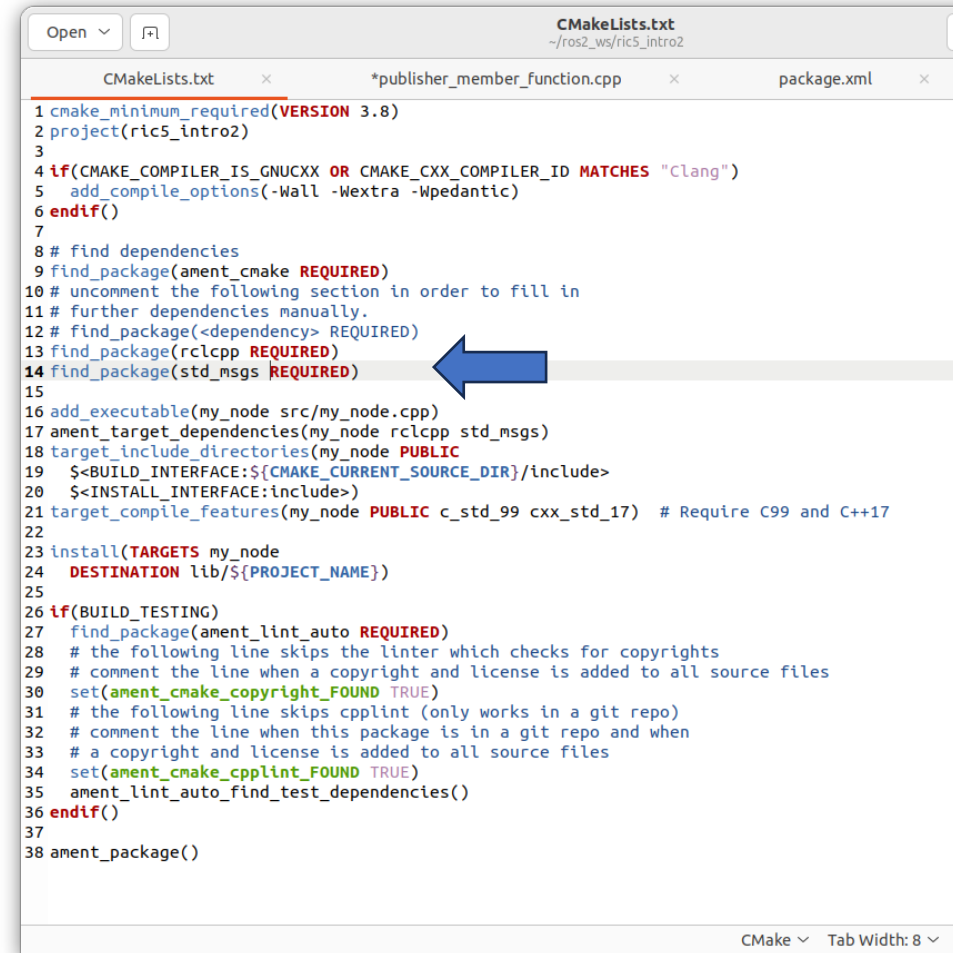
Meta information: version, author, dependencies etc.

- CMakeLists.txt

The Information required for the compiler

- /src

Your code goes here



```
1 cmake_minimum_required(VERSION 3.8)
2 project(ric5_intro2)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13 find_package(rclcpp REQUIRED)
14 find_package(std_msgs REQUIRED)
15
16 add_executable(my_node src/my_node.cpp)
17 ament_target_dependencies(my_node rclcpp std_msgs)
18 target_include_directories(my_node PUBLIC
19   $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
20   $<INSTALL_INTERFACE:include>)
21 target_compile_features(my_node PUBLIC c_std_99 cxx_std_17) # Require C99 and C++17
22
23 install(TARGETS my_node
24   DESTINATION lib/${PROJECT_NAME})
25
26 if(BUILD_TESTING)
27   find_package(ament_lint_auto REQUIRED)
28   # the following line skips the linter which checks for copyrights
29   # comment the line when a copyright and license is added to all source files
30   set(ament_cmake_copyright_FOUND TRUE)
31   # the following line skips cpplint (only works in a git repo)
32   # comment the line when this package is in a git repo and when
33   # a copyright and license is added to all source files
34   set(ament_cmake_cpplint_FOUND TRUE)
35   ament_lint_auto_find_test_dependencies()
36 endif()
37
38 ament_package()
```

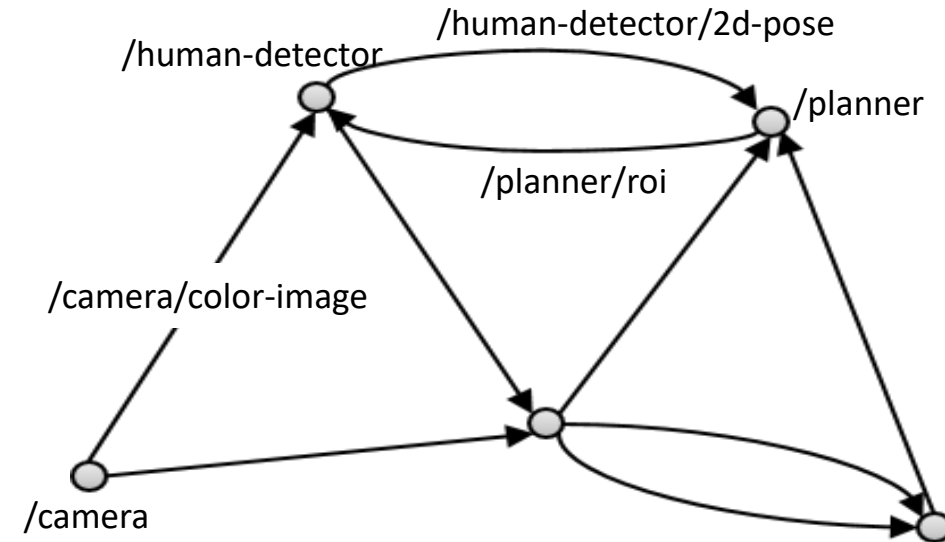
Workspace, launching nodes (cont'd)

Running a single node:

```
ros2 run my_package my_node [args]
```

Renaming, allows running multiple instances of the same program:

```
ros2 run my_package camera __name:=camera1
```



Workspace, launching nodes (cont'd)

Running a single node:

```
ros2 run my_package my_node [args]
```

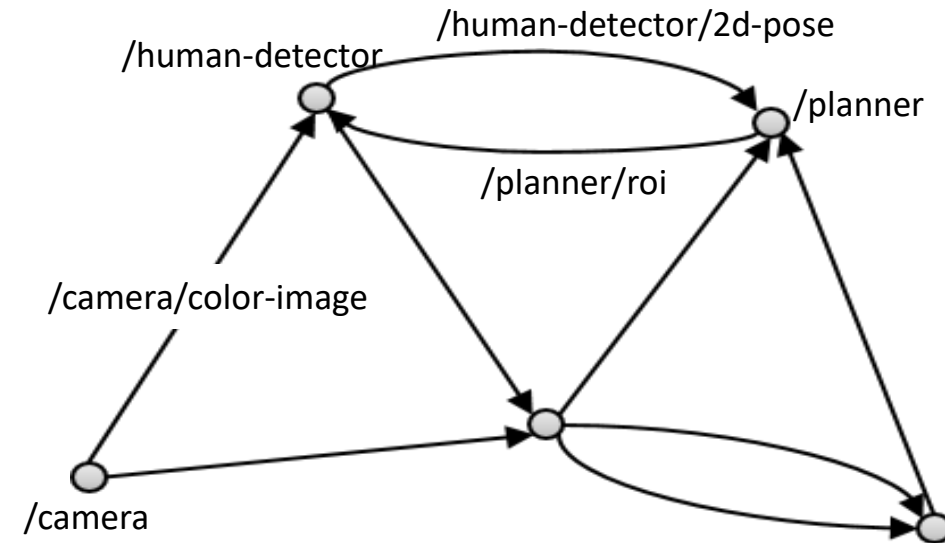
Renaming, allows running multiple instances of the same program:

```
ros2 run my_package camera __name:=camera1
```

Launching a bunch of nodes

```
ros2 launch my_package my_launchfile
```

This allows you to combine nodes rather freely



```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim1"/>
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim2"/>
  <node pkg="turtlesim" exec="mimic" name="mimic">
    <remap from="/input/pose" to="/turtlesim1/turtle1/pose"/>
    <remap from="/output/cmd_vel" to="/turtlesim2/turtle1/cmd_vel"/>
  </node>
</launch>
```

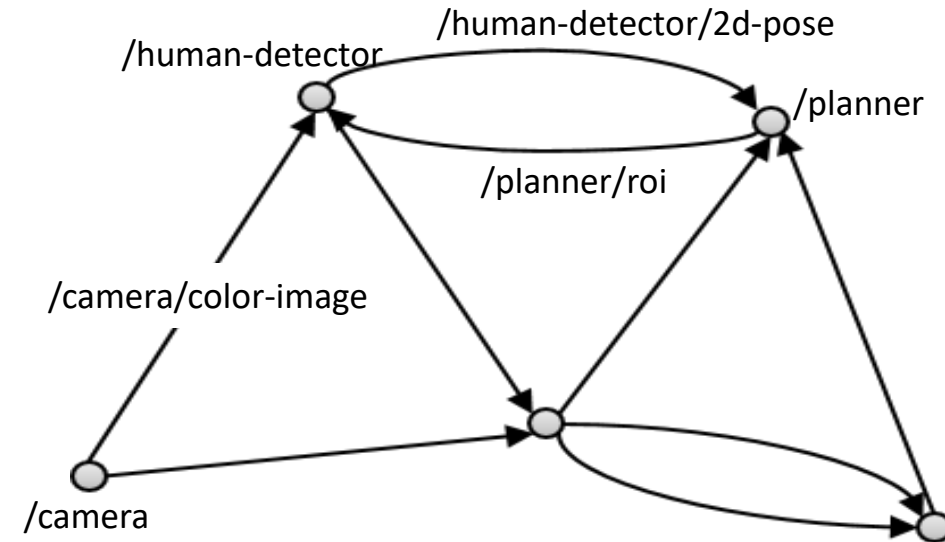
ROS Fundamentals: Topics

Messages:

- *Publisher/Subscriber* scheme
- Publisher announces (aka *advertises*) a *topic* which other nodes can subscribe to
- Handled asynchronously
 - *Callback function* at the receiver
- Via TCP/IP
 - Easy to distribute across devices
- Can be explored / debugged at runtime:

`ros2 topic list <topicname>` shows all topics and
`ros2 topic echo list <topicname>` prints the
content of a topic

`ros2 topic hz / type / pub / ...`



ROS Fundamentals: Topics

Messages:

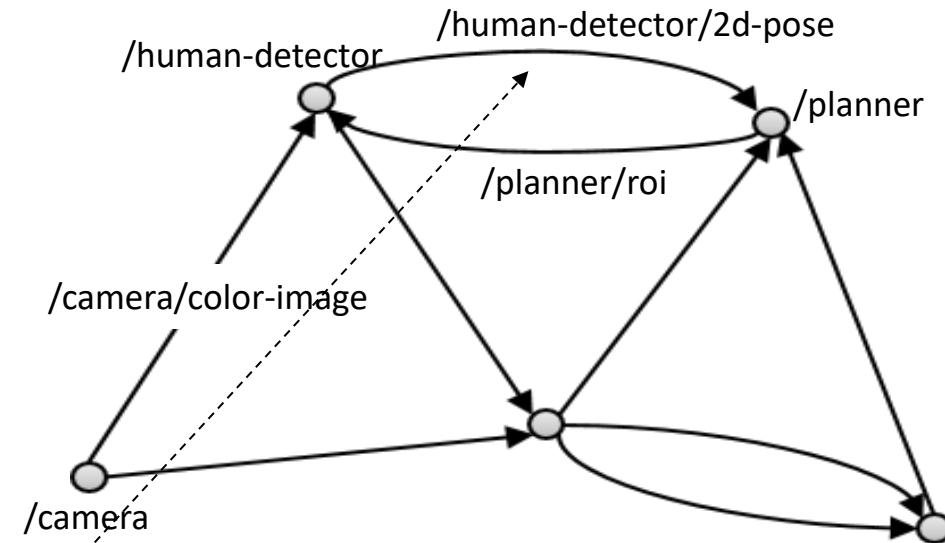
- *Publisher/Subscriber* scheme
- Publisher announces (aka *advertises*) a *topic* which other nodes can subscribe to
- Handled asynchronously
 - *Callback function* at the receiver
- Via TCP/IP
 - Easy to distribute across devices
- Can be explored / debugged at runtime:

`ros2 topic list <topicname>` shows all topics and
`ros2 topic echo list <topicname>` prints the
content of a topic

`ros2 topic hz / type / pub / ...`

Person.msg

```
int16 cpr
string full_name
float64 x
float64 y
```



ROS Fundamentals: Topics

Messages:

- *Publisher/Subscriber* scheme
- Publisher announces (aka *advertises*) a *topic* which other nodes can subscribe to
- Handled asynchronously
 - *Callback function* at the receiver
- Via TCP/IP
 - Easy to distribute across devices
- Can be explored / debugged at runtime:

`ros2 topic list <topicname>` shows all topics and
`ros2 topic echo list <topicname>` prints the
content of a topic

`ros2 topic hz / type / pub / ...`

Person.msg

```
int16 cpr
string full_name
float64 x
float64 y
```

inbuilt types

Primitive Type	Serialization	C++	Python (2/3)
bool (1)	unsigned 8-bit int	uint8_t (2)	bool
int8	signed 8-bit int	int8_t	int
uint8	unsigned 8-bit int	uint8_t	int (3)
int16	signed 16-bit int	int16_t	int
uint16	unsigned 16-bit int	uint16_t	int
int32	signed 32-bit int	int32_t	int
uint32	unsigned 32-bit int	uint32_t	Int
int64	signed 64-bit int	int64_t	long int
uint64	unsigned 64-bit int	uint64_t	long int
float32	32-bit IEEE float	float	Float
float64	64-bit IEEE float	Double	Float
string	ascii string (4)	std::string	str / bytes
time	secs/nsecs 32-bit units	ros::Time	rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration	rospy.Duration

<http://wiki.ros.org/msg>

ROS Fundamentals: Topics

Messages:

- *Publisher/Subscriber* scheme
- Publisher announces (aka *advertises*) a *topic* which other nodes can subscribe to
- Handled asynchronously
 - *Callback function* at the receiver
- Via TCP/IP
 - Easy to distribute across devices
- Can be explored / debugged at runtime:

`ros2 topic list <topicname>` shows all topics and
`ros2 topic echo list <topicname>` prints the
content of a topic

`ros2 topic hz / type / pub / ...`

inbuilt types

Primitive Type	Serialization	C++	Python (2/3)
bool (1)	unsigned 8-bit int	uint8_t (2)	bool
int8	signed 8-bit int	int8_t	int
uint8	unsigned 8-bit int	uint8_t	int (3)
int16	signed 16-bit int	int16_t	int
uint16	unsigned 16-bit int	uint16_t	int
int32	signed 32-bit int	int32_t	int
uint32	unsigned 32-bit int	uint32_t	Int
int64	signed 64-bit int	int64_t	long int
uint64	unsigned 64-bit int	uint64_t	long int
float32	32-bit IEEE float	float	Float
float64	64-bit IEEE float	Double	Float
string	ascii string (4)	std::string	str / bytes
time	secs/nsecs 32-bit units	ros::Time	rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration	rospy.Duration

<http://wiki.ros.org/msg>

PersonStamped.msg

Header header
int16 cpr
string full_name
Pose2D pose

Header.msg

#Standard metadata for higher-level flow data types
#Two-integer timestamp that is expressed as:
* stamp.secs: seconds (stamp_secs) since epoch
* stamp.nsecs: nanoseconds since stamp_secs
time-handling sugar is provided by the client library
time stamp
#Frame this data is associated with
string frame_id

geometry_msgs/Pose2D.msg

float64 x
float64 y
float64 theta

ROS Fundamentals: Topics (cont'd)

Imagine:

A mobile robot that is part of a fleet, e.g. in an airport, where a server occasionally dispatches new tasks.

Identify at least one challenge



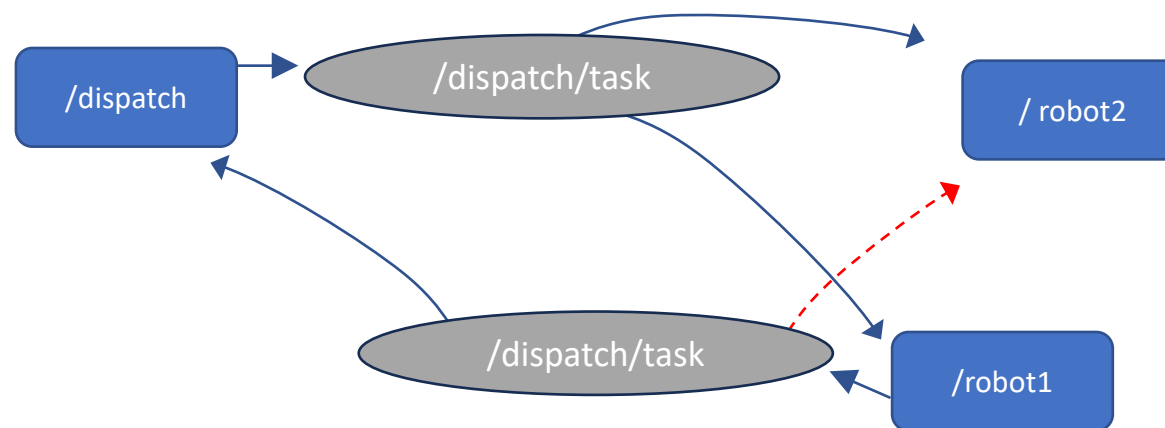
`/robots/accept_task`



Assumption:

- 1) as task-msg contains all data needed for a given task
- 2) accept_task-msg indicates that a robot takes the task

ROS Fundamentals: **Topics** (cont'd)

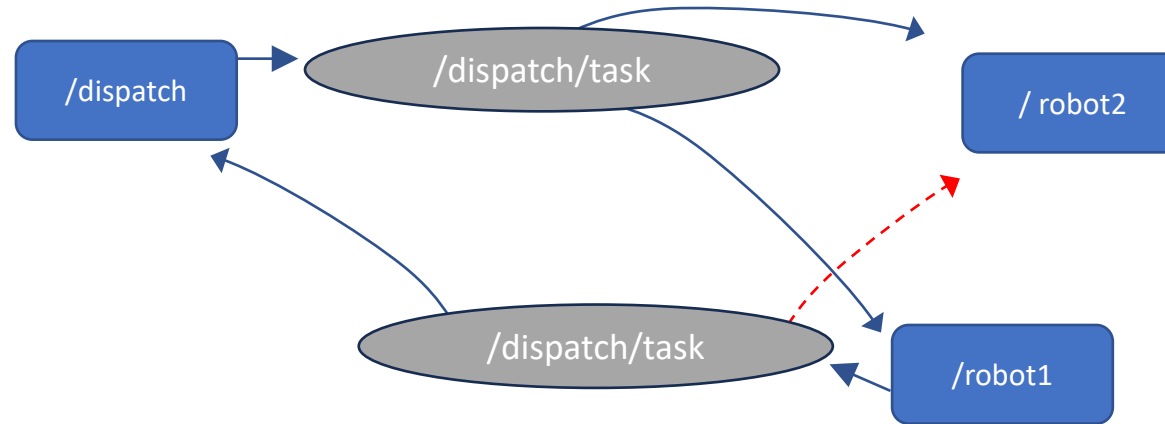


ROS Fundamentals: Topics (cont'd)

In general:

If you miss a message,
you will not get it.

In ROS1 the message could be *latched*.



```
uint32_t queue_size = 5;  
bool latched = true;  
  
ros::Publisher pub = nh.advertise<std_msgs::String>  
("topic_name", queue_size, latched);
```

!OBSOLETE!





ROS Fundamentals: Topics (cont'd)

In general:

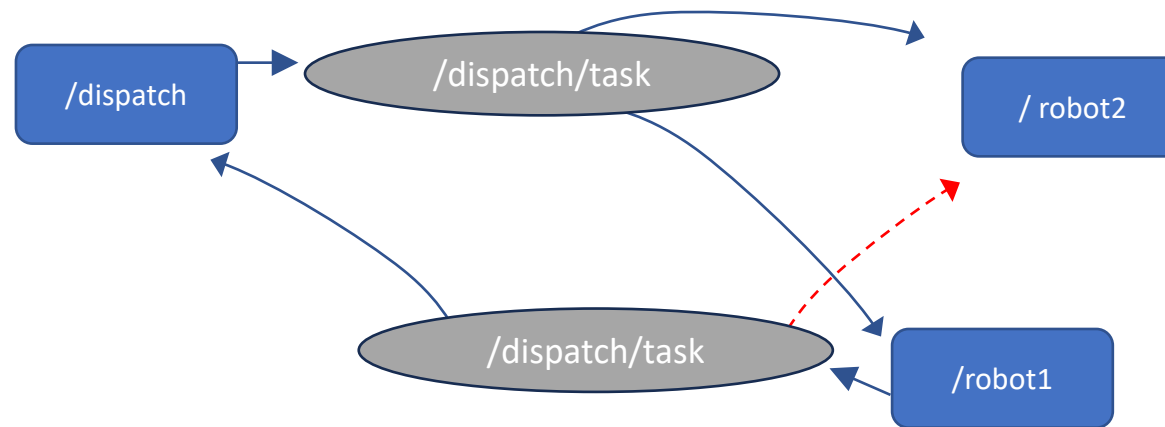
If you miss a message,
you will not get it.

~~In ROS1 the message could be *latched*.~~

In ROS2 solves this vis QoS

• ROS 1:

Publisher	Subscriber	Connection	Result
Not Latched	--	Yes	New Messages
Latched	--	Yes	New and Old Messages



• ROS 2:

Publisher	Subscriber	Connection	Result
Volatile	Volatile	Yes	New Messages
Volatile	Transient local	No	No Communication
Transient local	Volatile	Yes	New Messages
Transient local	Transient local	Yes	New and Old Messages

```
ros2 topic info <topic_name> --verbose
```

ROS Fundamentals: Services & Parameters

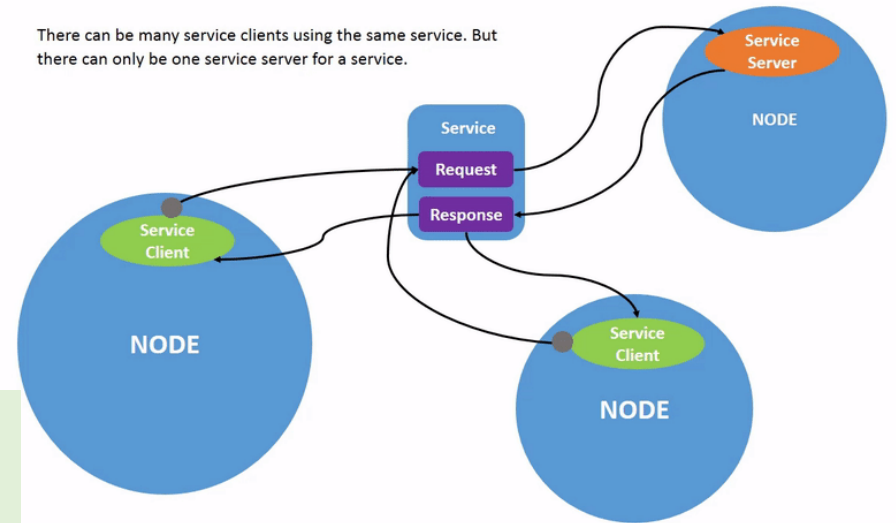
Services

- Each node can provide services
- Service calls are handled synchronously
 - Client is *blocked* until the call is completed and returns
- Can be explored, e.g.:
 - `ros2 service list`
 - (similar to topics)

CalculatePose.srv

JointState joints

Pose2D pose

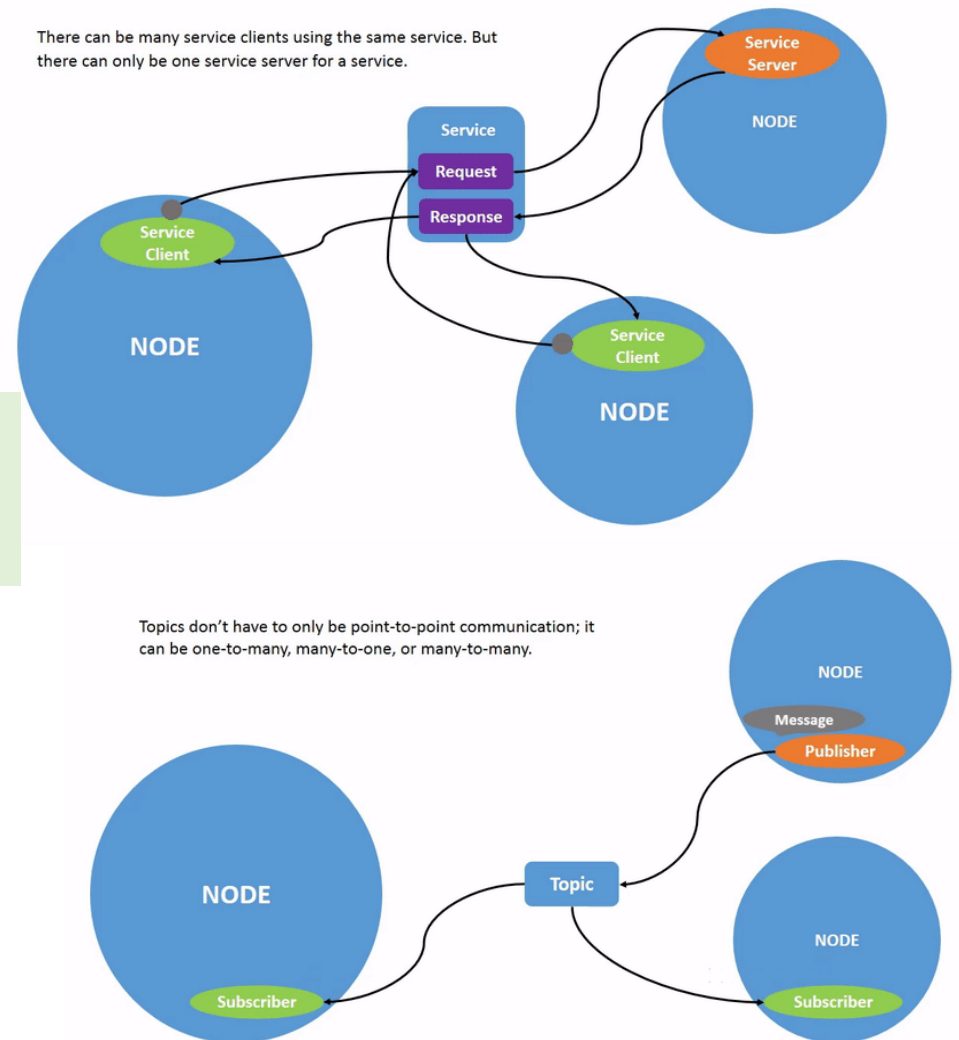


ROS Fundamentals: Services & Parameters

Services

- Each node can provide services
- Service calls are handled synchronously
 - Client is *blocked* until the call is completed and returns
- Can be explored, e.g.:
 - `ros2 service list`
 - (similar to topics)
- Services offer one-2-one communication
- Topics provide one-2-many (or many to many) communication

```
CalculatePose.srv
JointState joints
---
Pose2D pose
```



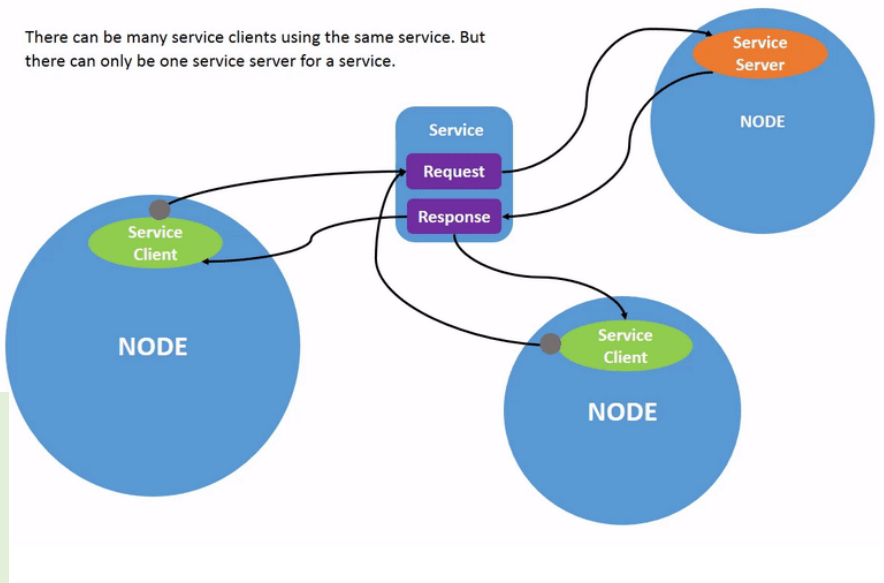
ROS Fundamentals: Services & Parameters

Services

- Each node can provide services
- Service calls are handled synchronously
 - Client is *blocked* until the call is completed and returns
- Can be explored, e.g.:
 - `ros2 service list`
 - (similar to topics)

Parameters

- Ideal for data that remains constant, e.g. names, IPs etc.
- Stored in .yaml file
- Loaded and accessed:
 - `ros2 param list / dump`
 - `ros2 param set <node> <parameter_name> <value>`
 - `ros2 param load <node> <parameter_file_name>`





ROS Tools

Rosbag

Recording and playing data

`ros2 bag record / play`

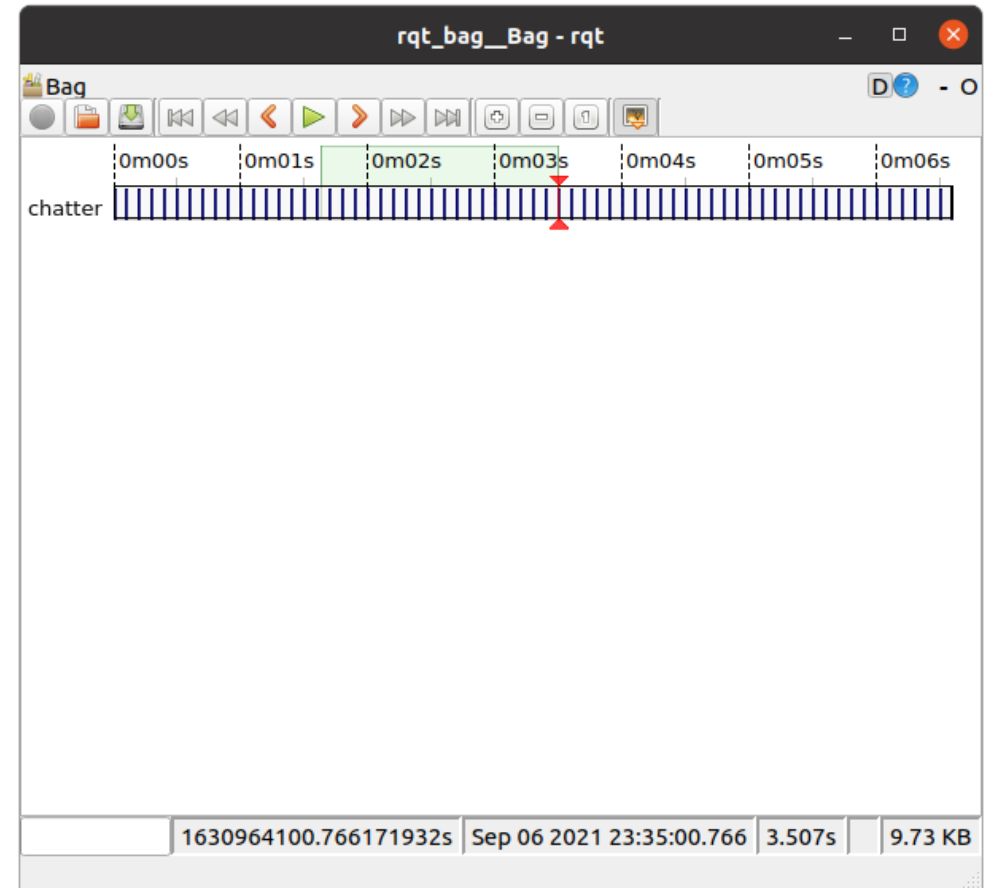
Specify what topics to record

- Can quickly fill up your harddrive
- Compression
- File system limitations
- Bandwidth limitations

rqt_bag can be used for inspecting rosbags

Limitations:

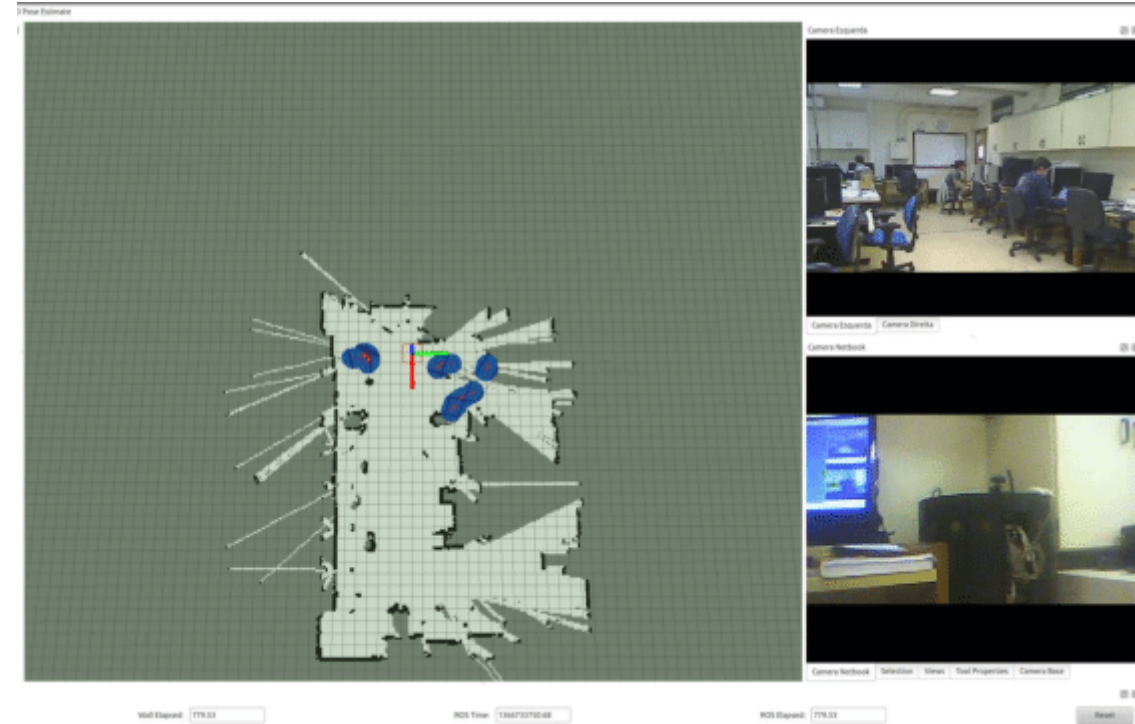
- Service calls
- Timing not perfect – can influence ability to reproduce the exact behaviour



Ros Visualization - RVIZ

`ros2 run rviz2` (or just `rviz2`)

- Visualization tool, 2D/3D
- Yet another rosnode
 - Subscribes to topics

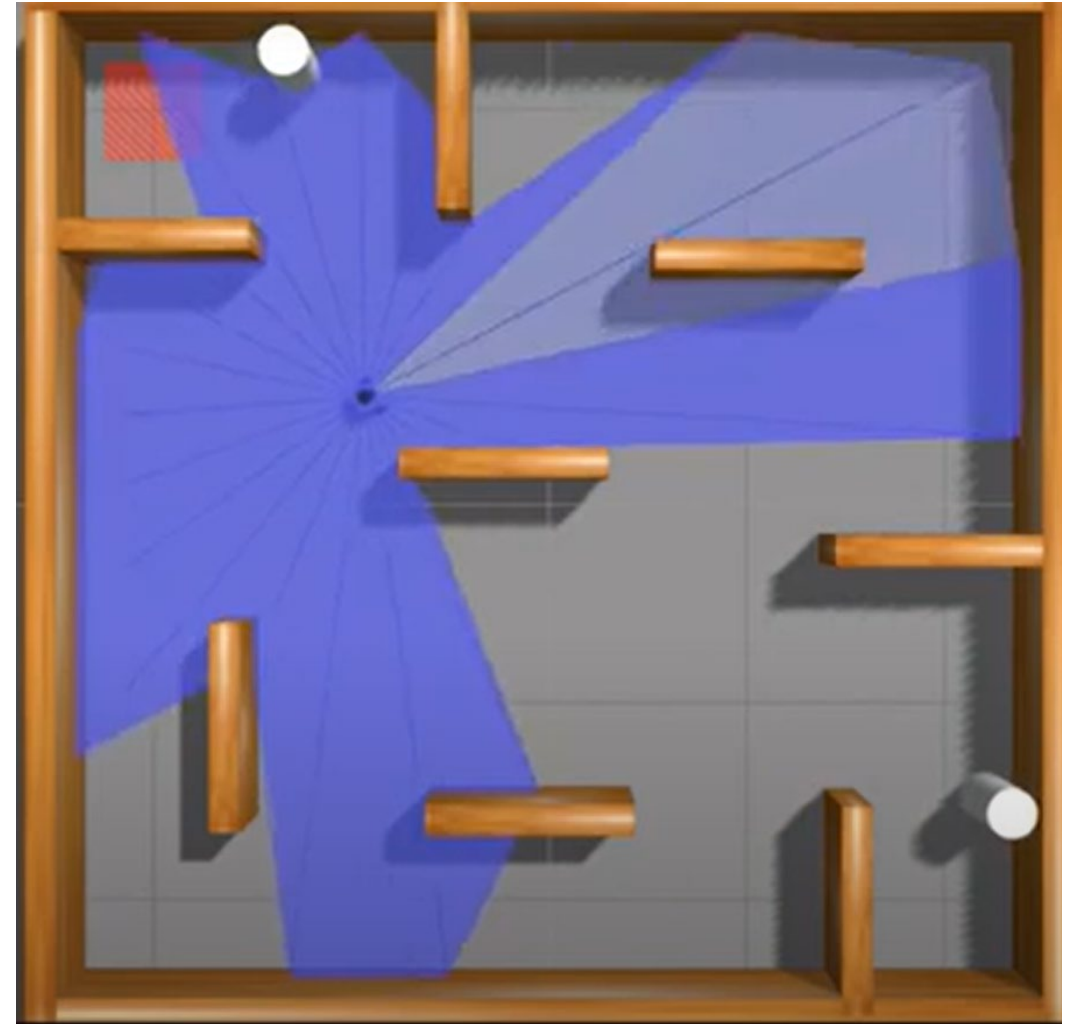


Simulation - Gazebo

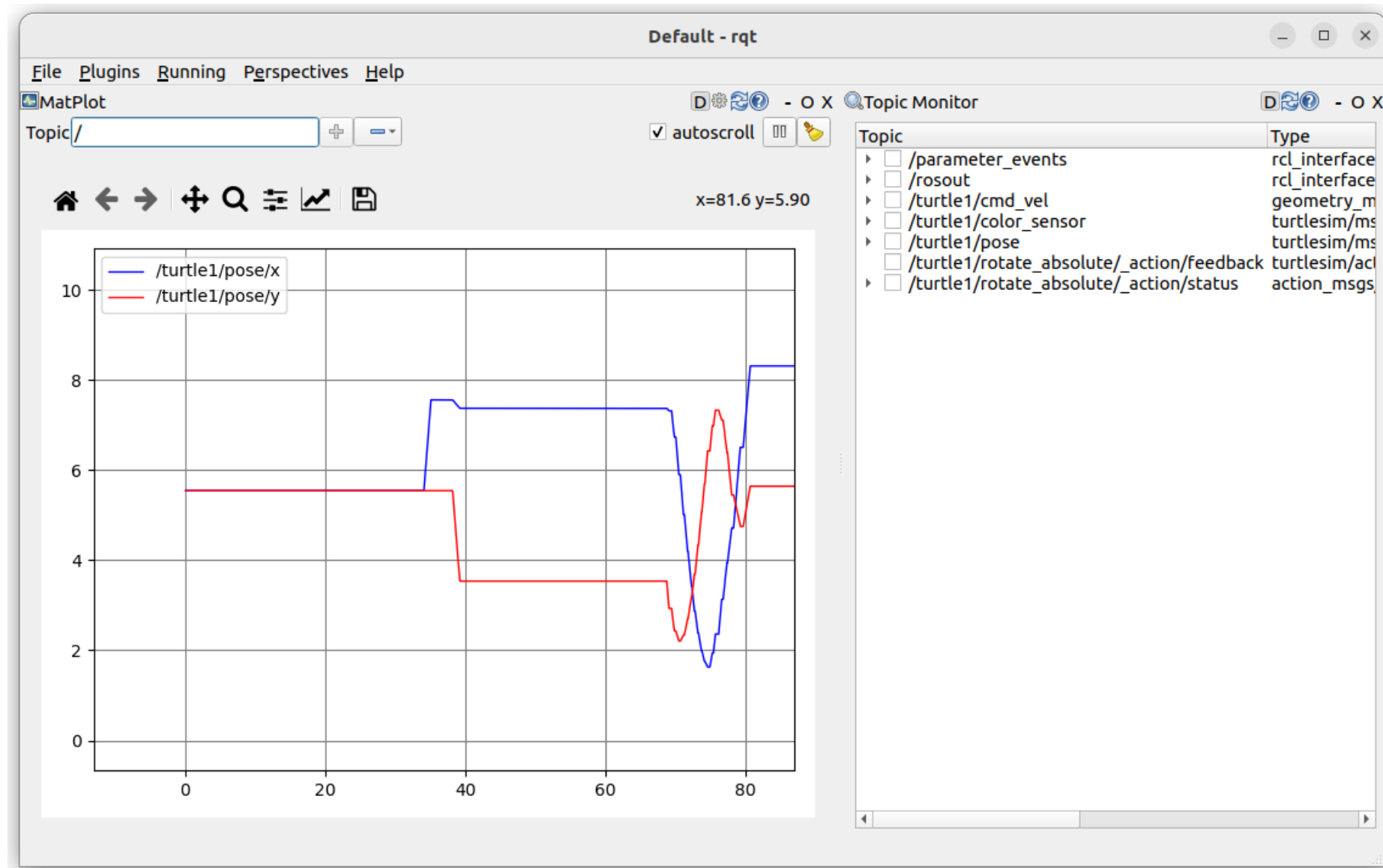
- Simulates 3D rigid body dynamics
- Simulates sensors, incl. noise
- 3D scene visualization
- Interface to ROS
- Various robot models exist (shared)
- Environments (worlds) can easily be set up (and shared)



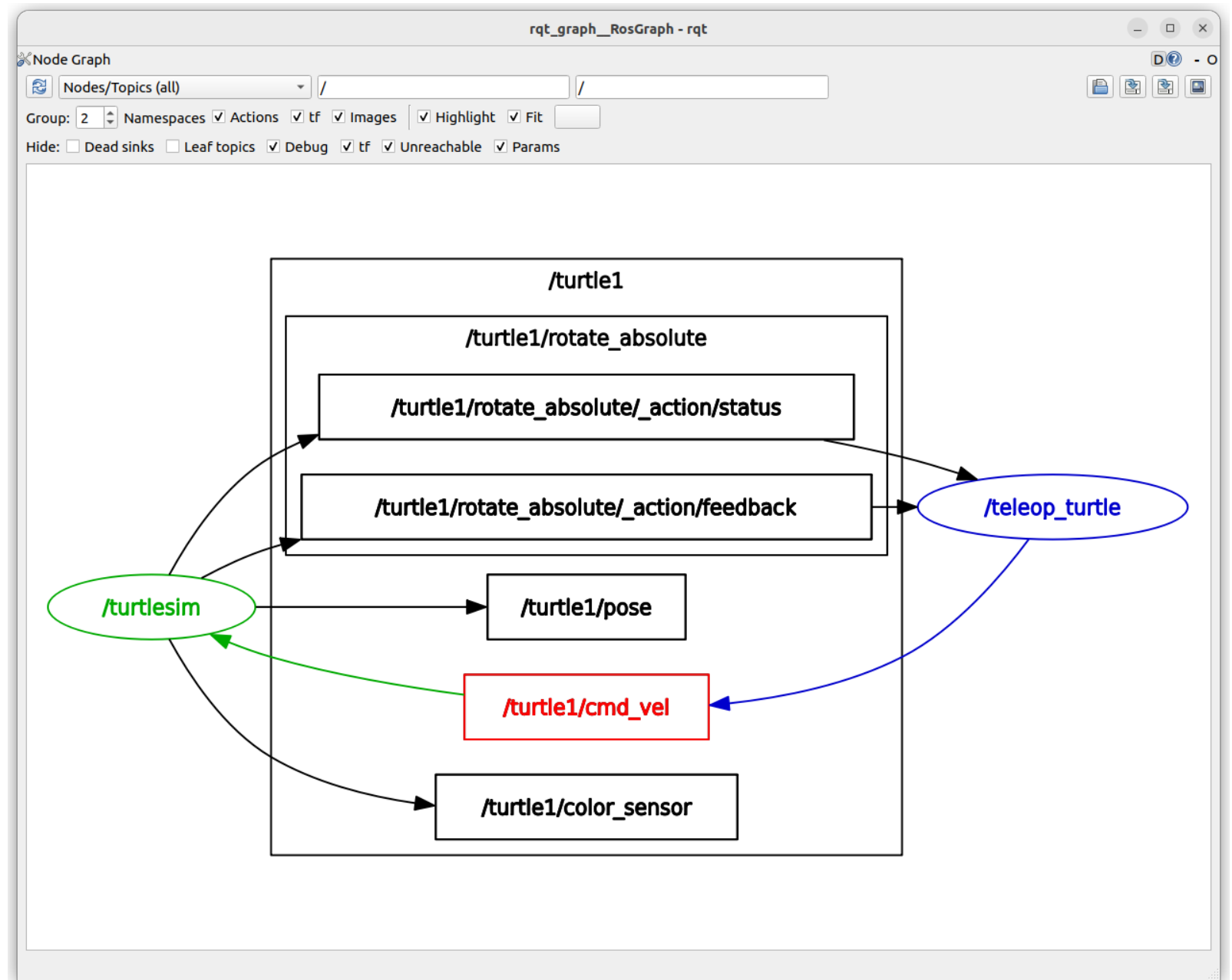
GAZEBO



RQT

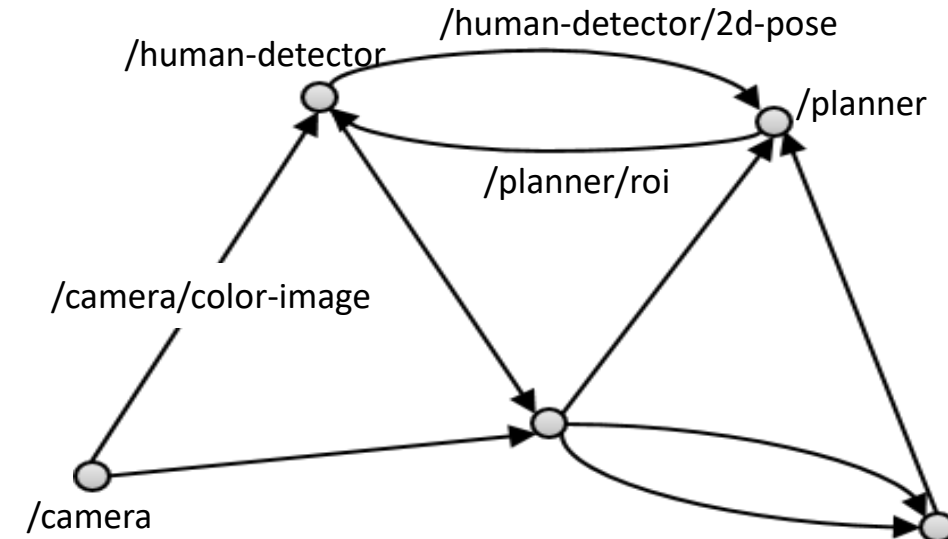
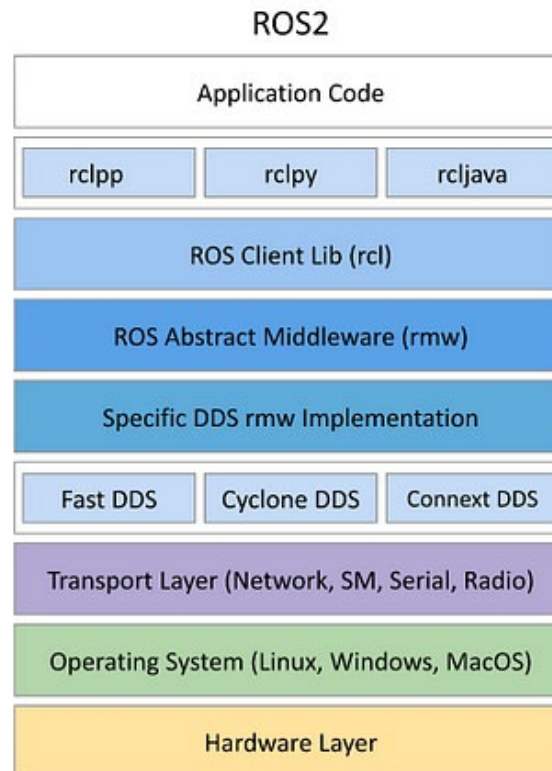


RQT_graph



Inter-node Communication in ROS

Can you see any drawback in the way communication between nodes is designed?



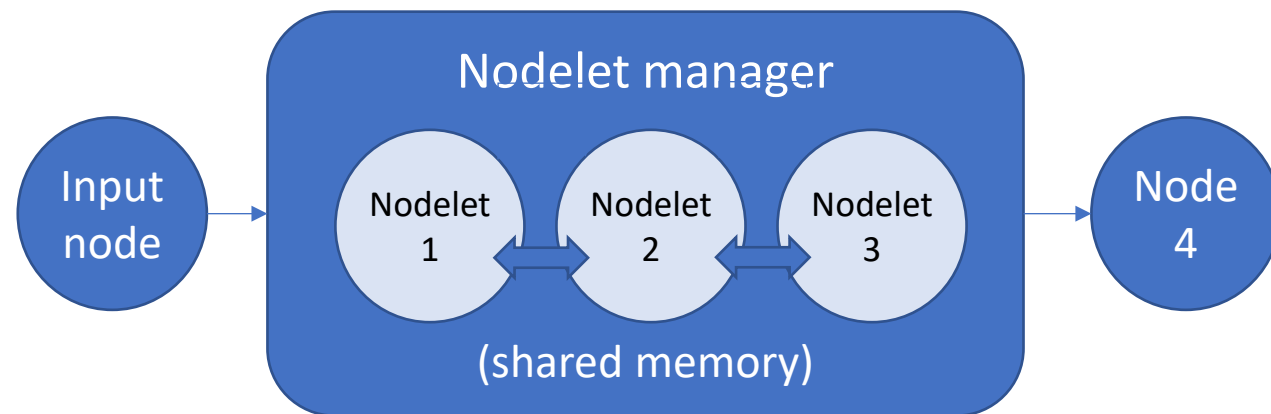
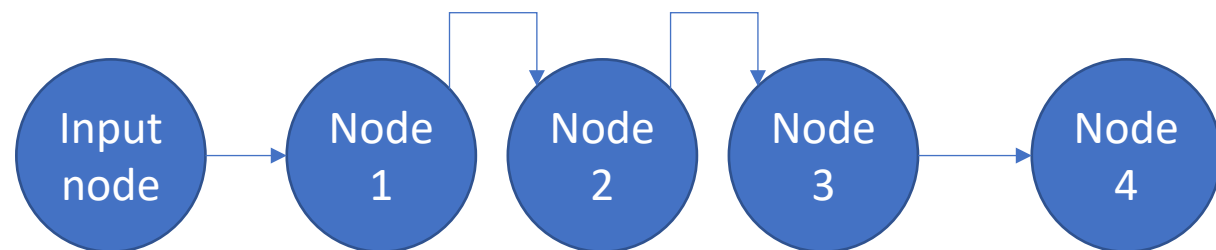
Nodelets / components

Nodes can be converted to nodelets which can access shared memory directly.

Avoids sharing data via the IP-stack which implies serialization.

Typically used to increase efficiency. Increases also complexity for the programmer.

ROS2 package:
ros-jazzy-intra-process-demo



Different Flavours of ROS

- **ROS 1**, aka ~~ROS~~



- **ROS Industrial** – focus on code quality, robustness etc. for automation
- **ROS 2**, version 2.0 of ROS, better real-time performance, rosbag has improved (services), encryption possible
General advice: When reading the documentation, check that the version, e.g., *jazzy*, matches with the OS
- **Micro-ros**, designed for microcontrollers, compatible with ROS 2

(some) ROS2 aspects not covered today

Topics that we don't expect to cover:

- Quality of Service, QoS
Enhancing performance on infrastructure with limited bandwidth
- ROS 2 Security
Encryption of communication, authentication etc.
- `ros2 doctor`
Checks your system and can detect potential problems
- Rosbridge
Allows combining ROS1 and ROS2

(some) ROS2 aspects not covered today

Topics that we don't expect to cover:

- Quality of Service, QoS
Enhancing performance on infrastructure with limited bandwidth
- ROS 2 Security
Encryption of communication, authentication etc.
- `ros2 doctor`
Checks your system and can detect potential problems
- Rosbridge
Allows combining ROS1 and ROS2

Later lectures:

- Gazebo (simulation tool)
- Modeling of robots
- TF2
- ROS Navigation stack



Take home from today and follow up

Keywords:

- Workspace
- colcon
- Package
- Rosrun and roslaunch
- Topics
- Services
- Messages
- rqt graph
- Domain ID
- RVIZ
- Rosbag
- Namespace
- Rosparam
- TF2 (will be revisited)
- (nodelets)

Exercises

- Aim: explore key concepts

There are many tutorials available on <https://ros.org>

- ‘Sidequest’: become operational in the first place

Scope of lecture

- Base line: You need to *use* ROS to learn it, lectures cannot substitute that
 - Common issue: source your distro!
- Aim: Provide an overview that gets you started and enables you to look up details

Programming Exercise 1